

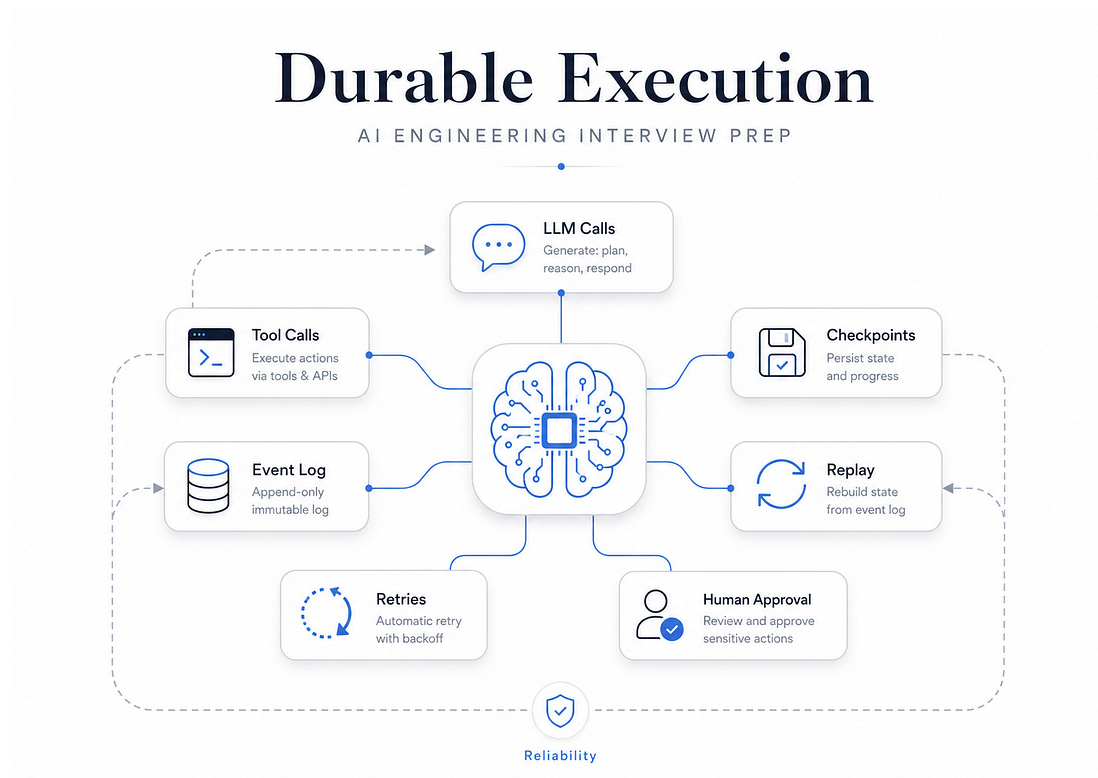
Long-Running Background Agents and Durable Execution

Production-grade AI agents cannot rely on fragile while True loops; they need durable workflows, checkpoints, replay, retries, and human-in-the-loop recovery.



AI ENGINEERING INSIDER

MAY 31, 2026



In production AI systems, especially long-running agents, failure is expected. The agent may run for minutes or hours, call tools, invoke LLMs, wait for human approval, write files, update databases, or call APIs. If the process crashes halfway, a naive agent restarts from scratch and may repeat expensive or dangerous actions.

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed

A durable agent avoids this by treating each LLM call, tool call, API request, or file operation as an **activity**. Each activity is logged before and after execution. When the system crashes, the workflow replays from the event log and returns completed activity results without re-executing them.

The best interview answer:

A production long-horizon AI agent should be modeled as a deterministic workflow that orchestrates non-deterministic activities. LLM calls, tool calls, and side-effect operations should be logged, checkpointed, retried, and replayed so the system can recover safely without duplicate side effects or repeated cost.

Concepts You Must Know

Durable Execution

Durable execution means the agent can survive crashes, restarts, deployments, and human delays. The workflow state is persisted, so the system can resume from the latest checkpoint rather than starting over.

Example:

- ↳ Bad design: Agent runs in memory using `while True`
- ↳ Good design: Agent runs as a durable workflow with event logs and checkpoints

Workflow

A workflow is deterministic orchestration logic.

It decides:

- ↳ What activity runs next
- ↳ What branch to take
- ↳ When to wait
- ↳ When to resume
- ↳ When to stop
- ↳ When to request human approval

The workflow itself should not directly call random APIs, generate timestamps, call LLMs, or execute side effects. Those belong inside activities.

Activity

An activity is a non-deterministic unit of work.

Examples:

- ↳ LLM call
- ↳ Tool call
- ↳ File write
- ↳ HTTP request
- ↳ Database update
- ↳ Email send
- ↳ Payment operation
- ↳ GitHub PR creation
- ↳ Code execution
- ↳ Human approval request

Activities should be logged with:

- ↳ Input
 - ↳ Start time
 - ↳ Completion status
 - ↳ Output
 - ↳ Error
 - ↳ Retry count
 - ↳ Idempotency key
-

Event Log

The event log is the source of truth for recovery.

It records:

- ↳ Workflow started
- ↳ Activity scheduled
- ↳ Activity completed
- ↳ Activity failed
- ↳ Retry attempted
- ↳ Human approval requested
- ↳ Human approval received
- ↳ Workflow resumed
- ↳ Workflow completed

During replay, the system reads the event log and skips activities that have already been completed.

Replay

Replay means the workflow code runs again from the beginning, but completed activities are not rerun. Their previous results are returned from the durable log.

This prevents:

- ↳ Duplicate LLM billing
- ↳ Duplicate tool calls
- ↳ Duplicate emails
- ↳ Duplicate database writes
- ↳ Duplicate user approvals
- ↳ Duplicate external side effects

Checkpoint

A checkpoint is a saved state of a workflow. In agent systems, checkpoints are often scoped by `thread_id`, session ID, workflow ID, or run ID.

A checkpoint may include:

- ↳ Current step
- ↳ Conversation state
- ↳ Tool outputs
- ↳ Pending human approval
- ↳ Budget usage
- ↳ Retry state
- ↳ Intermediate artifacts
- ↳ Memory state
- ↳ Last completed activity

`thread_id`

`thread_id` identifies a specific agent session.

For example:

```
thread_id = "customer-support-ticket-123"
thread_id = "autonomous-coding-run-456"
```

```
thread_id = "research-agent-session-789"
```

This prevents two sessions from mixing logs, checkpoints, or tool outputs.

Why LLM Calls Need Durable Execution

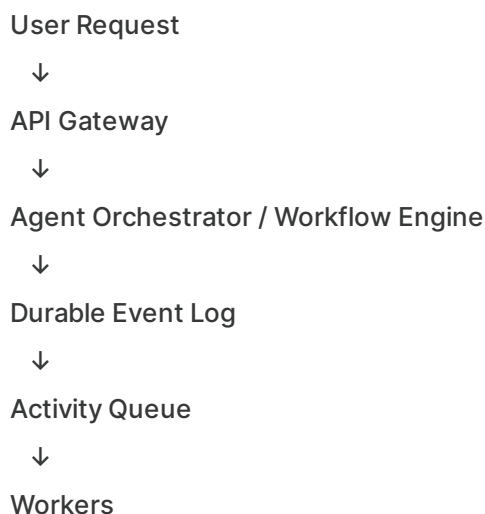
LLM calls are risky because they are:

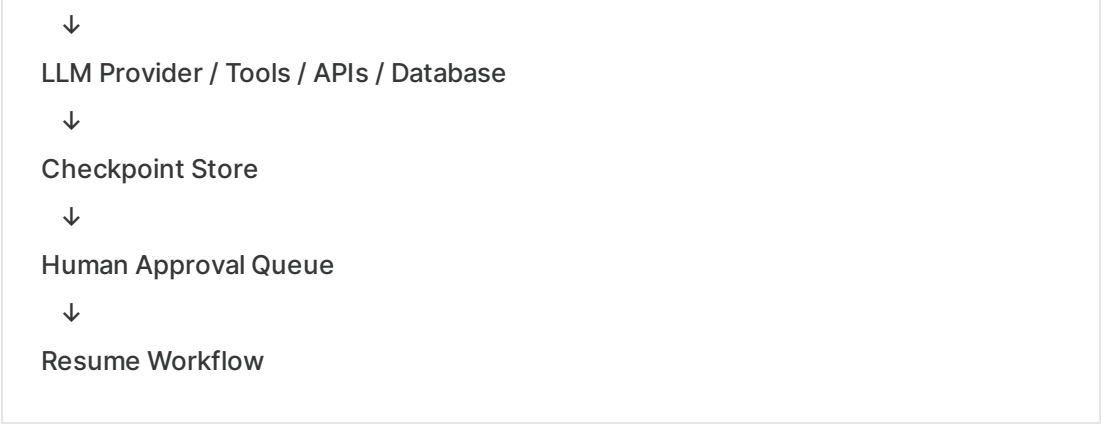
- ↳ Non-deterministic
- ↳ Expensive
- ↳ Slow
- ↳ Rate-limit prone
- ↳ Tool-triggering
- ↳ Sensitive to model/version changes
- ↳ Hard to reproduce exactly

In production, each LLM call should be treated as an activity, not just a regular function call.

Production Architecture

A strong interview architecture for durable AI agents:





Recommended Components

Layer	Purpose
API Gateway	Receives user request
Workflow Engine	Controls deterministic orchestration
Event Log	Stores every workflow transition
Activity Queue	Runs LLM/tool/API tasks asynchronously
Worker Pool	Executes activities
Checkpoint Store	Stores latest workflow state
Human Approval Queue	Pauses and resumes workflows
Budget Guardrail	Stops runaway cost
Observability	Logs traces, metrics, retries, failures
Idempotency Store	Prevents duplicate side effects

Backend Choices

Backend	Best Use
PostgreSQL	Durable checkpoint storage, auditability, production workflows
Redis	Fast state/cache, but needs persistence enabled
SQLite	Local development only
Cloudflare Durable Objects	Session-scoped distributed workflows
Temporal/Cadence	Full workflow orchestration
LangGraph checkpointing	Agent state persistence
Queue + DB	Lightweight custom durable system

When Durable Execution Is Necessary

Use durable execution when:

- ↳ Agent runs longer than a few minutes
- ↳ Agent uses tools with side effects
- ↳ Agent calls expensive LLMs repeatedly
- ↳ Agent waits for human approval
- ↳ Agent performs autonomous coding
- ↳ Agent writes to databases/files
- ↳ Agent sends emails or notifications
- ↳ Agent executes business workflows
- ↳ Agent must survive deploys/restarts
- ↳ Agent needs audit trails

When Durable Execution May Be Overkill

Durable execution may be unnecessary when:

- ↳ Task is short-lived
- ↳ Task is read-only
- ↳ No side effects exist

- ↳ No human approval is needed
 - ↳ The output can be regenerated cheaply
 - ↳ The whole task must complete inside one context window
-

Production Design Answer Template

Use this in interviews:

I would not build a long-running AI agent as an in-memory loop. I would separate deterministic orchestration from non-deterministic activities. The workflow would control the sequence of steps, while LLM calls, tool calls, API requests, and file writes would be modeled as activities. Each activity would be logged with inputs and outputs, retried safely, and protected with idempotency keys. The workflow state would be checkpointed by `thread_id`, so if the process crashes or the system redeploys, it can replay completed steps from the event log and only execute missing activities. For human approvals, I would persist a waiting state and resume from that exact point after approval. I would also add budget limits, timeout policies, retry policies, observability, and HITL-on-resume for high-risk actions.

Mock Interview Q&A

Q1. What is durable execution in AI agents?

Durable execution is a design pattern where a long-running AI workflow persists its state, activity history, and checkpoints so it can recover after crashes, restarts, or deploys. Instead of restarting from scratch, the agent replays completed steps from the event log and continues from the last safe point.

A strong answer:

Durable execution makes long-running agents recoverable. Every non-deterministic operation, such as an LLM call or tool call, is treated as an activity and recorded in a durable log. On restart, completed activities are replayed from stored results instead of being executed again.

Q2. Why is a while True loop dangerous for production AI agents?

A while True loop keeps state in memory. If the host crashes, all progress is lost. The agent may repeat tool calls, duplicate side effects, re-bill LLM calls, and ask the user for approvals again.

Better answer:

A while True loop is acceptable for a prototype, but unsafe for production long-horizon agents. It has no durable state, no replay semantics, no idempotency guarantees, and no reliable recovery model.

Q3. What is the difference between a workflow and an activity?

A workflow is deterministic orchestration code. It controls the sequence of execution.

An activity is a non-deterministic operation, such as an LLM call, API request, tool execution, file write, or human approval request.

Interview answer:

The workflow decides what should happen. The activity does the actual work. The workflow must be deterministic so it can replay

safely, while activities can be non-deterministic because their inputs and outputs are logged.

Q4. Why should LLM calls be modeled as activities?

LLM calls are expensive, non-deterministic, slow, and failure-prone. If an agent crashes after an LLM call, the system should not call the model again unless necessary.

Strong answer:

I model LLM calls as activities because they are non-deterministic and expensive. By logging their inputs and outputs, I can replay the result after recovery without re-billing or introducing a different model response.

Q5. What is replay in durable execution?

Replay is the recovery process where the workflow code runs again, but previously completed activities return stored outputs from the event log instead of executing again.

Example:

Original run:

Step 1: LLM call completed

Step 2: Tool call completed

Step 3: Crash before email send

Replay:

Step 1: Return stored LLM result

Step 2: Return stored tool result

Step 3: Execute email send

Q6. How do you prevent duplicate side effects?

Use idempotency keys and activity logs.

For example:

- ↳ Email send uses email_request_id
- ↳ Payment uses payment_idempotency_key
- ↳ Database write uses unique transaction ID
- ↳ File write uses a deterministic file path/version
- ↳ The tool calls stores the completion status before retry

Interview answer:

I would make every side-effect activity idempotent. The activity should have a stable idempotency key, and the external system should reject or safely ignore duplicates.

Q7. What should be stored in a checkpoint?

A checkpoint should store enough state to resume safely.

Include:

- ↳ thread_id
- ↳ Current workflow step
- ↳ Conversation state
- ↳ Tool results
- ↳ LLM outputs
- ↳ Pending approval state
- ↳ Retry metadata

- ↳ Budget usage
 - ↳ Last completed activity
 - ↳ Intermediate artifacts
-

Q8. Why is `thread_id` important?

`thread_id` scopes the workflow state to a specific user session, agent run, ticket, or task.

Without `thread_id`, multiple agent sessions may collide or read the wrong checkpoint.

Strong answer:

`thread_id` is the durable session key. It lets the system resume the correct workflow state and prevents event logs from different agent sessions from mixing.

Q9. How would you design human-in-the-loop approval for a durable agent?

I would persist the workflow in a `WAITING_FOR_HUMAN_APPROVAL` state. The approval request would be stored externally with a request ID. When the user approves or rejects, the workflow resumes from that exact checkpoint.

Example flow:

```
graph TD; A[Agent proposes action] --> B[Workflow stores approval request];
```

Workflow pauses



Human approves



Approval event is recorded



Workflow resumes



Activity executes

Q10. What happens if an approval arrives after the system redeploys?

The workflow should resume from the persisted checkpoint. Since the waiting state is durable, the approval event can be matched to the correct `thread_id` and approval request ID.

Good interview phrase:

Human approval should not depend on a live process. It should be represented as durable workflow state.

Q11. What is deterministic workflow logic?

Deterministic workflow logic produces the same decisions during replay.

Avoid inside workflow code:

↳ `datetime.now()`

↳ `random.random()`

↳ Direct LLM calls

↳ Direct API calls

↳ Direct database writes

- ↳ Environment-dependent branching
- ↳ Non-versioned prompts
- ↳ Unstable model outputs

Put those inside activities instead.

Q12. How do workflow engines handle non-determinism?

They isolate nondeterminism within activities or specialized side-effect APIs. For example, instead of calling the wall clock directly inside workflow code, the engine may provide a deterministic `Workflow.now()` that returns a replay-safe timestamp.

Q13. What are the main failure modes of long-running agents?

Key failure modes:

- ↳ Host crash
 - ↳ Deployment restart
 - ↳ LLM timeout
 - ↳ Tool failure
 - ↳ API rate limit
 - ↳ Duplicate side effect
 - ↳ Lost human approval
 - ↳ Context drift
 - ↳ Budget runaway
 - ↳ Infinite loop
 - ↳ Corrupted checkpoint
 - ↳ Wrong replay behavior
 - ↳ Non-deterministic workflow divergence
-

Q14. What is HITL-on-resume?

HITL-on-resume means the system may require fresh human approval when resuming after a crash, especially before high-risk actions.

Example:

- ↳ Low-risk replay: continue automatically
- ↳ High-risk replay: ask a human to confirm before sending an email, deploying code, or charging a payment

Q15. How do you handle retry logic?

Use bounded retries with exponential backoff and clear failure policies.

Example:

LLM timeout:

Retry 3 times with exponential backoff

Payment API failure:

Retry only if idempotency key exists

Human approval timeout:

Escalate or cancel

Tool failure:

Mark activity failed and pause workflow

Q16. How do you control costs in a long-running agent?

Use budget guardrails.

Track:

- ↳ Total tokens
- ↳ Number of LLM calls
- ↳ Cost per workflow
- ↳ Wall-clock time
- ↳ Tool execution count
- ↳ Retry count
- ↳ Maximum planning loops
- ↳ Maximum autonomous actions

Strong answer:

Durable execution makes longer runs possible, but budget controls make them safe.

Q17. How would you design a 6-hour autonomous coding agent?

Architecture:

- ↳ User submits task
- ↳ Workflow creates `thread_id`
- ↳ Agent plans work
- ↳ Checkpoint after plan
- ↳ Agent edits code in small activities
- ↳ Checkpoint after each file change
- ↳ Agent runs tests
- ↳ Checkpoint test results
- ↳ Agent summarizes diff
- ↳ Human approval before PR

- ↳ Agent opens PR
- ↳ Workflow completes

Fresh HITL required before:

- ↳ Pushing code
 - ↳ Creating PR
 - ↳ Running destructive commands
 - ↳ Changing production config
 - ↳ Installing dependencies
 - ↳ Accessing secrets
 - ↳ Deploying
-

Q18. What observability would you add?

Add traces and metrics for:

- ↳ Workflow duration
 - ↳ Activity latency
 - ↳ LLM cost
 - ↳ Retry count
 - ↳ Failure rate
 - ↳ Replay count
 - ↳ Checkpoint age
 - ↳ Human approval wait time
 - ↳ Tool-call success rate
 - ↳ Idempotency conflicts
 - ↳ Budget exhaustion
 - ↳ Stuck workflows
-

Q19. What is the difference between checkpointing and replay?

Checkpointing stores the latest state.

Replay reconstructs execution by running the workflow against the event log.

Simple distinction:

- ↳ Checkpoint = saved state
- ↳ Replay = recovery process
- ↳ Event log = historical truth

Q20. What is a strong production-grade answer for durable AI agents?

Use this answer in interviews:

For production long-running AI agents, I would use durable execution. The workflow should be deterministic and persisted by thread_id. LLM calls, tool calls, API calls, and side effects should be modeled as activities with durable input/output logs. On crash, the workflow replays completed activities from the event log and only executes missing work. I would add idempotency keys for side effects, bounded retries, exponential backoff, checkpoint storage, HITL approval states, observability, and budget kill switches. This design prevents duplicate side effects, reduces wasted LLM cost, and makes autonomous agents recoverable.

System Design Interview

Design a durable, long-running AI research agent that can browse documents, call LLMs, write summaries, ask humans for approval, and resume after crashes.

Expected Answer Structure

- ↳ Requirements
 - ↳ Failure modes
 - ↳ Architecture
 - ↳ Data model
 - ↳ Workflow lifecycle
 - ↳ Activity design
 - ↳ Checkpointing strategy
 - ↳ HITL strategy
 - ↳ Retry/idempotency
 - ↳ Observability
 - ↳ Tradeoffs
-

Sample System Design Answer

I would design the agent as a durable workflow system.

The user request creates a `thread_id`. The workflow stores the initial state in a checkpoint store. The agent then schedules activities for document retrieval, LLM summarization, tool calls, and writing outputs. Each activity has an activity ID, input hash, status, output, retry count, and idempotency key.

The workflow itself remains deterministic. It only decides the next step based on persisted results. LLM calls and tool calls are never executed directly inside the workflow. They run as activities through workers.

If the system crashes, the workflow engine reloads the event log for the `thread_id`, replays completed activities, and resumes from the last incomplete step. If the next action is high risk, such as sending a report or writing to a shared repository, the workflow enters a durable human-approval state.

For reliability, I would add retry policies, exponential backoff, dead-letter queues, budget limits, timeout policies, idempotency protection, and full observability.

Simple Data Model

```
CREATE TABLE workflow_runs (  
  thread_id TEXT PRIMARY KEY,  
  status TEXT,  
  current_step TEXT,  
  checkpoint_json JSONB,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);
```

```
CREATE TABLE activity_events (  
  event_id TEXT PRIMARY KEY,  
  thread_id TEXT,  
  activity_id TEXT,  
  activity_type TEXT,  
  input_hash TEXT,  
  status TEXT,  
  output_json JSONB,  
  error TEXT,  
  retry_count INT,  
  idempotency_key TEXT,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);
```

```
CREATE TABLE human_approvals (  
  approval_id TEXT PRIMARY KEY,  
  thread_id TEXT,
```

```
requested_action TEXT,  
status TEXT,  
requested_at TIMESTAMP,  
responded_at TIMESTAMP  
);
```

Python Pseudocode

```
def run_workflow(thread_id, user_request):  
    state = load_checkpoint(thread_id)  
  
    plan = run_activity(  
        thread_id=thread_id,  
        activity_id="create_plan",  
        fn=call_llm,  
        input={"request": user_request}  
    )  
  
    research = run_activity(  
        thread_id=thread_id,  
        activity_id="retrieve_sources",  
        fn=search_documents,  
        input={"plan": plan}  
    )  
  
    summary = run_activity(  
        thread_id=thread_id,  
        activity_id="write_summary",  
        fn=call_llm,  
        input={"research": research}  
    )  
  
    approval = request_human_approval(
```

```
        thread_id=thread_id,  
        action="send_report",  
        payload=summary  
    )  
  
    if approval == "approved":  
        run_activity(  
            thread_id=thread_id,  
            activity_id="send_report",  
            fn=send_email,  
            input={"summary": summary},  
            idempotency_key=f"send_report:{thread_id}"  
        )  
  
    mark_workflow_complete(thread_id)
```

Interview Red Flags

Avoid saying:

- ↳ "I would just keep the agent running in a loop."
- ↳ "If it crashes, I restart from the beginning."
- ↳ "LLM calls are deterministic if temperature is zero."
- ↳ "We do not need idempotency."
- ↳ "Human approval can stay in memory."
- ↳ "Retries solve everything."
- ↳ "Redis is always durable by default."
- ↳ "Replay means running everything again."

This Substack is reader-supported. To
receive new posts and support my work,
consider becoming a free or paid subscriber.

✓ Subscribed